Programmazione 2

a.a. 2023-2024

correttezza, invariante di ciclo
con esempi su ordinamento di array

Correttezza di un programma

Una funzione deve funzionare

Cosa ci si aspetta da una funzione?

```
tipo-ritorno funz(tipo1 x1, …, tipok xk) {
      … codice di funz …
}
```

Una funzione deve funzionare

Cosa ci si aspetta da una funzione?

```
tipo-ritorno funz(tipo1 x1, …, tipok xk) {
      … codice di funz …
}
```

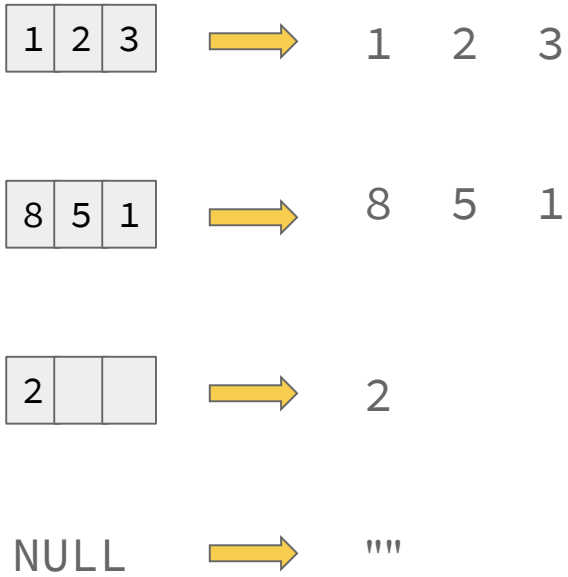
CHE FACCIA QUANTO STABILITO DALLA SUA SPECIFICA

Esempio: dato un array stampa tutti i suoi valori

Esempio

Dato un array di interi, la funzione STAMPA visualizza tutti i valori in esso contenuti

```
void stampa(int *arr) { … }
```



SI PUÒ SBAGLIARE IN MOLTI MODI

sbagliare logica
es. devo calcolare la somma e calcolo la differenza

sbagliare sintassi
es. dimentico una parentesi graffa

sbagliare controllo
es. condizione di uscita da un loop errata o un richiamo ricorsivo

saltare un caso
es. dimentico di gestire la stringa vuota

SI PUÒ SBAGLIARE IN MOLTI MODI

sbagliare logica
es. devo calcolare la somma e calcolo la differenza

sbagliare sintassi
es. dimentico una parentesi graffa

saltare un caso
es. dimentico di gestire la stringa vuota

in questo caso tradiamo la specifica. La funzione non fa quanto atteso perché abbiamo realizzato il calcolo sbagliato, es. per ignoranza del dominio

SI PUÒ SBAGLIARE IN MOLTI MODI

sbagliare sintassi
es. dimentico una parentesi graffa

sbagliare logica

questo errore è meno grave perché il compilatore ce lo segnalerà e sarà facile da correggere

saltare un caso
es. dimentico di gestire la stringa vuota

sbagliare controllo
es. condizione di uscita da un loop errata o un richiamo ricorsivo

SI PUÒ SBAGLIARE IN MOLTI MODI

questi invece sono errori di programmazione veri e propri, cioè riguardano la capacità di programmare

possono avere effetti gravissimi

sbagliare sintassi
es. dimentico una parentesi graffa

saltare un caso
es. dimentico di gestire la stringa vuota

sbagliare controllo
es. condizione di uscita da un loop errata o un richiamo ricorsivo

Non è un programmatore ...

chi si fa guidare dal compilatore (es. GCC)

chi si fa guidare dal debugger (es. GDB)

è importante **avere il controllo** di quello che si realizza

Idealmente

- ★ Scrivo la funzione su carta
- ★ La copio in un file facendo attenzione alla sintassi
- ★ Compilo: nessun errore sintattico!
- ★ Eseguo e:
  - tutti i possibili casi sono gestiti
  - l’output è sempre corretto e
  - la funzione termina!

IL CODICE VA PROGETTATO

Progettazione delle funzioni: passi di base

- ★ Scrivere (in un testo) eventuali assunzioni sui dati ricevuti dalla funzione
- ★ Identificare i possibili casi da trattare e scrivere un esempio per ciascuno
- ★ Se la funzione contiene cicli:
  scrivere quale condizione è vera ad ogni iterazione
- ★ Scrivere la condizione che sarà vera (sui dati e sull’eventuale output prodotto) al termine dell’esecuzione

Esempio: funzione che effettua una divisione

- ★ **Assunzioni iniziali**: assumo che la funzione riceva due valori, dividendo e divisore, e che il divisore sia diverso da zero
- ★ **Descrizione del codice che esplica la funzione**: effettua il calcolo (semplice, un’istruzione sola)
- ★ **Condizione sul risultato**: se l’assunzione iniziale è vera il risultato sarà il valore prodotto dal primo parametro fratto il secondo parametro

Alternativa: funzione che effettua una divisione

- ★ **Assunzioni iniziali**: assumo che la funzione riceva due valori, dividendo e divisore, e di non avere garanzie che il divisore sia diverso da zero
- ★ **Descrizione del codice che esplica la funzione**: effettua il calcolo (semplice, un’istruzione sola)
- ★ **Condizione sul risultato**: quando il divisore è diverso da 0 il risultato sarà il valore prodotto dal primo parametro fratto il secondo parametro, altrimenti restituisco il dividendo e un messaggio di errore

Scrittura per raffinamenti successivi

- ★ coerentemente con le assunzioni fatte, si scrive **sotto forma di commenti** quel che farà la funzione
- ★ si trasformano via via i commenti in codice
- ★ alla fine si prova il codice sull'insieme di casi precedentemente identificati

Si può guidare la scrittura corretta?

- ★ Quando il codice è lineare (semplice sequenza di calcoli) gli unici errori sono di tipo **sintattico** e di tipo **logico**

Si può guidare la scrittura corretta?

★ Quando il codice è lineare (semplice sequenza di calcoli) gli unici errori sono di tipo **sintattico** e di tipo **logico**

COMPILATORE

TEST SUITE

Test Suite

Codice di prova:

- ★ richiede di avere identificato i **possibili casi** a cui la funzione è applicata e avere identificato un **esempio** per ciascuno
- ★ gli esempi saranno utilizzati in altrettante **invocazioni della funzione**, per esempio in un main
- ★ devono essere corredati di codice di output (a terminale o su file) che mostri all'utente **il risultato o l'eventuale errore**

Si può guidare la scrittura corretta?

Quando il codice è strutturato o fa dei richiami ricorsivi:

- ★ Bisogna garantire la **terminazione** della funzione (non vi sono esecuzioni infinite)
- ★ Bisogna garantire che **ogni passo** (iterazione) dell'esecuzione rispetti una **proprietà di correttezza** (viene fatto quanto previsto dalla specifica)

C'È UN MODO PER GUIDARE LA SCRITTURA CORRETTA?

- ★ Bisogna garantire la **terminazione** della funzione (non vi sono esecuzion

TERMINAZIONE

- ★ Bisogna garantire che ogni passo (iterazione) dell'esecuzione rispetti una **proprietà di correttezza** (viene fatto c ca)

CORRETTEZZA PARZIALE

CORRETTEZZA

Correttezza & co.

La correttezza non sostituisce ma si aggiunge alla verifica della correttezza logica e sintattica del programma

Correttezza parziale

vale prima

```
FUNZIONE(x1, …, xn) {
    … calcola …
    return y;
}
```

vale dopo

PREDICATO DI INPUT

P_IN(x1, …, xn)

proprietà che vale sui dati che saranno elaborati dalla funzione

PREDICATO DI OUTPUT

P_OUT(x1, …, xn, y)

proprietà che vale sui dati elaborati e l'output prodotto

Correttezza parziale di una funzione

Indichiamo con **fz(t_1 p_1, …, t_n p_n)** una **funzione**

Tale funzione ha n **parametri** p_1, … p_n,

rispettivamente di **tipo** t_1, … t_n

Esempio:

quadrato(int x) ha un solo parametro di tipo intero

Correttezza parziale di una funzione 1/3

fz(t_1 p_1, …, t_n p_n) è **parzialmente corretta** quando:

$\forall$ $<x_1, \ldots, x_n>$

ennupla di valori rispettivamente di tipo $t_1, \ldots, t_n$,

…

Esempio: per quadrato(int x)

i valori saranno tutti i possibili numeri interi

0, 1, 2, 3, …

Correttezza parziale di una funzione 2/3

… se vale che:

1. **P-IN(x_1,...,x_n)** è vera
2. l'esecuzione di fz(x_1,...,x_n) **termina restituendo y**,

allora **P-OUT(x_1,...,x_n,y)** è vera

Correttezza parziale di una funzione 3/3

1. **P-IN(x_1,...,x_n)**: è una proprietà dell'input

2. **P-OUT(x_1,...,x_n,y)**: è una proprietà lega l'input e l'output

Esempio: per quadrato(int x)

P_IN(x): x è un numero intero,

P_OUT(x, y): è y == x*x

Terminazione di una funzione

Una funzione fz(x_1,...,x_n) è **terminante** se

∀ <x_1, …, x_n> tali per cui **P-IN(x_1,...,x_n)** è vera,

l'esecuzione di fz(x_1,...,x_n) si conclude.

Esempio: per quadrato(int x)

quando P_IN(x) è vera (input valido), quadrato ha fine

Correttezza

Una funzione fz(x_1,...,x_n) è **corretta** se valgono due condizioni:

1. è parzialmente corretta
2. è terminante

Esempio: calcolare il valore minimo di un array

Scriviamo un primo canovaccio

Esempio: calcolare il valore minimo di un array

- ★ Assumo di ricevere un array di dimensione N (N>0)
- ★ Il valore restituito sarà uno dei valori contenuti nell’array, il più piccolo
- ★ definisco una variabile di comodo dello stesso tipo degli elementi dell’array
- ★ la inizializzo opportunamente
- ★ uso un ciclo che scorre l’array
- ★ a ogni iterazione la variabile conterrà il valore più piccolo fra quelli visti al momento
- ★ per decidere se aggiornarla la confronto con l’elemento corrente
- ★ alla fine restituisco il valore contenuto nella variabile di comodo

Esempio: calcolare il valore minimo di un array

```
/*

   P_IN: Assumo di ricevere un array di dimensione N (N>0)

   P_OUT: Il valore restituito sarà uno dei valori contenuti
          nell'array, il più piccolo

*/

int MINIMO(int a[], int N) {
```

- ★ definisco una variabile di comodo dello stesso tipo degli elementi dell'array
- ★ la inizializzo opportunamente
- ★ uso un ciclo che scorre l'array
- ★ a ogni iterazione la variabile conterrà il valore più piccolo fra quelli visti al momento
- ★ per decidere se aggiornarla la confronto con l'elemento corrente
- ★ alla fine restituisco il valore contenuto nella variabile di comodo

```
}
```

Esempio: calcolare il valore minimo di un array

```
/*

   P_IN: Assumo di ricevere un array di dimensione N

   P_OUT: Il valore restituito sarà uno dei valori contenuti
          nell'array, il più piccolo

*/

int MINIMO(int a[], int N) {

    int temp; // definisco una variabile di comodo …
```

- ★ la inizializzo opportunamente
- ★ uso un ciclo che scorre l'array
- ★ a ogni iterazione la variabile conterrà il valore più piccolo fra quelli visti al momento
- ★ per decidere se aggiornarla la confronto con l'elemento corrente
- ★ alla fine restituisco il valore contenuto nella variabile di comodo

```
}
```

Esempio: calcolare il valore minimo di un array

```
/*

   P_IN, P_OUT: … // descrizione

*/

int MINIMO(int a[], int N) {

     int temp; // definisco una variabile di comodo …

     int i;
```

★ la inizializzo opportunamente

```
     for ( i=0 ; i<N; i++) { // uso un ciclo che scorre l'array
```

★ a ogni iterazione la variabile conterrà il valore più piccolo fra quelli visti al momento
★ per decidere se aggiornarla la confronto con l'elemento corrente
★ alla fine restituisco il valore contenuto nella variabile di comodo

```
      }

}
```

Esempio: calcolare il valore minimo di un array

…

```
int MINIMO(int a[], int N) {
      int temp; // definisco una variabile di comodo …
      int i;
  ★   la inizializzo opportunamente
      for ( i=0 ; i<N; i++) { // uso un ciclo che scorre l'array
  ★   a ogni iterazione la variabile conterrà il valore più piccolo fra quelli visti al momento
        if (a[i] < temp) // per decidere se aggiornarla la confronto con l'elemento corrente
            temp = a[i];
  ★   alla fine restituisco il valore contenuto nella variabile di comodo
       }
}
```

Esempio: calcolare il valore minimo di un array

…

```
int MINIMO(int a[], int N) {
      int temp = a[0]; // definisco una variabile di comodo … e la inizializzo opportunamente
      int i;
      for ( i=0 ; i<N; i++) { // uso un ciclo che scorre l'array
  ★   a ogni iterazione la variabile conterrà il valore più piccolo fra quelli visti al momento
         if (a[i] < temp) // per decidere se aggiornarla la confronto con l'elemento corrente
             temp = a[i];
  ★   alla fine restituisco il valore contenuto nella variabile di comodo
       }
}
```

Esempio: calcolare il valore minimo di un array

…

```
int MINIMO(int a[], int N) {

      int temp = a[0]; // definisco una variabile di comodo … e la la inizializzo opportunamente

      int i;

      // a ogni iterazione la variabile conterrà il valore più piccolo
      // fra quelli visti al momento

      for ( i=0 ; i<N; i++) { // uso un ciclo che scorre l'array

         if (a[i] < temp) // per decidere se aggiornarla la confronto …

             temp = a[i];

       }

       return temp; // alla fine restituisco il valore contenuto nella variabile di comodo

}
```

Esempio: faccio qualche aggiustamento

…

```
int MINIMO(int a[], int N) {

      int temp = a[0]; // definisco una variabile di comodo … e la la inizializzo opportunamente

      int i;

      // a ogni iterazione la variabile conterrà il valore più piccolo
      // fra quelli visti al momento

      for ( i=1 ; i<N; i++) { // uso un ciclo che scorre l'array

         if (a[i] < temp) // per decidere se aggiornarla la confronto …

             temp = a[i];

       }

       return temp; // alla fine restituisco il valore contenuto nella variabile di comodo

}
```

Minimo è terminante?

La terminazione va dimostrata

- ★ Una sequenza termina sempre, se i suoi passi terminano
- ★ Un ciclo termina se iterazione dopo iterazione ci si avvicina alla condizione di uscita
- ★ Qualcosa di simile vale per le funzioni ricorsive

MINIMO È TERMINANTE?

La terminazione va dimostrata

- ★ MINIMO contiene un **ciclo for**
- ★ La condizione di uscita è che l’indice **1 (inizializzato a 0) diventi uguale a N**
- ★ Ad ogni iterazione l’indice **i viene incrementato di 1**
- ★ Quindi prima o poi renderà vera la condizione di uscita portando MINIMO a **terminare**

ATTENZIONE!

La terminazione va dimostrata guardando il **codice effettivo**, **non le intenzioni del programmatore**

- ★ Se per errore MINIMO contenesse questo ciclo for

  ```
  for ( i=0 ; i<N; ) { … }
  ```

- ★ non potremmo dire che i viene incrementata a ogni iterazione!

Altro esempio: cerca un elemento

Ricominciamo scrivendo il codice di una funzione che cerca se un elemento è presente in un array di N numeri interi

Se è presente (almeno una volta) restituisce l’indice di una cella che lo contiene, altrimenti restituisce -1

Altro esempio: cerca un elemento

Ricominciamo scrivendo il codice di una funzione che cerca se un elemento è presente in un array di N numeri interi

Se è presente (almeno una volta) restituisce l'indice **di una cella** che lo contiene, altrimenti restituisce -1

PERCHÈ restituisce l'indice di UNA CELLA e non DELLA CELLA?

Perché l'elemento può comparire più volte nell'array

Cerca: aspettative

P_IN: arr ha num. di elementi N > 0 (l’array è definito)

P_OUT: se (risultato >= 0) arr[risultato] == el

se (risultato == -1) arr[0]!=el AND … arr[N-1]!=el

```
int cerca(int arr[N], int el) {

}
```

Cerca: aspettative

```
int cerca(int arr[N], int el) {
   // percorro l’array iterativamente finché o trovo el
   // o arrivo in fondo
   // a ogni iterazione:
   // se il valore corrente == el memorizzo il suo indice
   // in una variabile (che deve valere -1 se non lo trovo)
   // ed esco dal ciclo
   // restituisco l’output
}
```

Cerca: aspettative

```
int cerca(int arr[N], int el) {
   // percorro l'array iterativamente finché o ho el o arrivo in fondo
   for (i=0; i<N && ?????; i++) {
     // a ogni iterazione:
     // se il valore corrente == el memorizzo il suo indice
     // in una variabile (che deve valere -1 se non lo trovo)
     // ed esco dal ciclo
   }
   // restituisco l'output
}
```

Cerca: aspettative

```
int cerca(int arr[N], int el) {
   int i, finito=0;
   // percorro l'array iterativamente finché o ho el o arrivo in fondo
   for (i=0; i<N && !finito; i++) {
     // se il valore corrente == el memorizzo il suo indice
     // …
   }
   // restituisco l'output
}
```

Cerca: aspettative

```
int cerca(int arr[N], int el) {
   int i, finito=0;
   for (i=0; i<N && !finito; i++) {
     if (arr[i] == el) { // se il valore corrente == el
        // memorizzo il suo indice in una variabile (che deve valere -1
        // se non lo trovo) ed esco dal ciclo
     }
   }
   // restituisco l'output
}
```

Cerca

```
int cerca(int arr[N], int el) {
   int i, finito=0; int ris = -1;
   for (i=0; i<N && !finito; i++) {
     if (arr[i] == el) { // se il valore corrente == el
        // memorizzo il suo indice in una variabile
        // (che deve valere -1 se non lo trovo) ed
        // esco dal ciclo
        ris = i;
        finito = 1;
     }
   }
   // restituisco l'output
}
```

Cerca: realizzazione

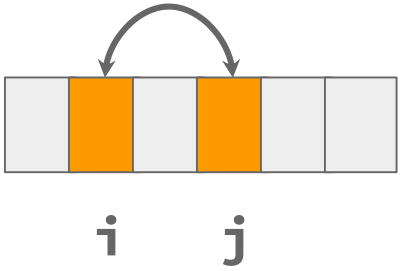
```
int cerca(int arr[N], int el) {
   int i, finito=0; int ris = -1;
   for (i=0; i<N && !finito; i++) {
     if (arr[i] == el) { // se il valore corrente == el
        // memorizzo il suo indice in una variabile (che deve valere -1
        // se non lo trovo) ed esco dal ciclo
        ris = i; finito = 1;
     }
   }
   // restituisco l'output
   return ris;
}
```

Termina?

Sì per lo stesso motivo dell’esempio precedente

Anzi se l’elemento di interesse è presente la terminazione potrà avvenire prima di avere percorso tutto l’array

Un po' di notazione

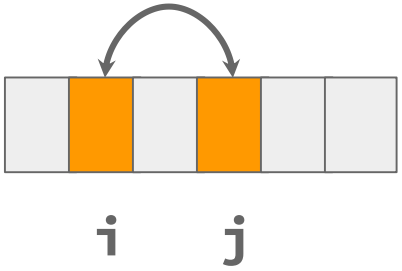


A volte nelle proprietà di input e di output occorre riferirsi **sia ai valori iniziali** di qualche variabile **sia ai valori che si hanno al termine dell'esecuzione**.

Esempio SCAMBIA:

```
void scambia(int[] arr,
             int i , int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

Un po' di notazione



Usiamo l'apice ' per indicare un valore iniziale

Esempio SCAMBIA:

P-IN(arr, i, j): i e j indici validi di arr

P-OUT(arr, i, j):

a[i] == a'[j] e a[j] == a'[i]

```
void scambia(int[] arr,
             int i , int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

Un po' di notazione

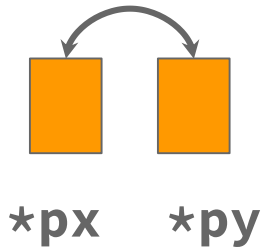
La notazione ’ (apice) si usa anche per i puntatori

Un po' di notazione

La notazione ' (apice) si usa anche con i puntatori

Esempio SCAMBIA_P:

```
void scambia_p(int *px , int *py) {
    int temp = *px;
    *px = *py;
    *py = temp;
}
```



Un po' di notazione

P-IN(px, py): true

P-OUT(px, py): *px == (*py)' e *py == (*px)'

Esempio SCAMBIA_P:

```
void scambia_p(int *px , int *py) {
    int temp = *px;
    *px = *py;
    *py = temp;
}
```

Warning!

In alcuni esercizi d’esame vi chiederemo di scrivere P_IN e P_OUT delle funzioni

→ Allenatevi a scrivere P_IN e P_OUT delle vostre funzioni

Progettazione di cicli guidata dalla invariante di ciclo (I.C.)

Come si sviluppa un ciclo che sia corretto per costruzione?

A cosa conviene pensare per scrivere un ciclo che sia corretto per costruzione?

- Per scrivere un ciclo non banale è indispensabile pensare subito al passo generico, NON al passo iniziale.
- Cioè bisogna pensare <u>non</u> agli estremi (inizio o fine del ciclo), bensì ad <u>un generico punto intermedio</u>.
- La generica situazione in un punto intermedio, precisata opportunamente, costituisce, come vedremo, ciò che si chiama l'invariante del ciclo.

In che ordine conviene scrivere il codice un ciclo?

~~1~~ *inizializzazione*

while (*condizione*) ~~2~~

corpo dell'iterazione ~~3~~

NO !

L'ordine giusto è l'opposto! (almeno in linea di principio)

1. Per scrivere l'*inizializzazione* si deve sapere cosa deve fare il ciclo
2. Per scrivere la *condizione* (*guardia*) si deve conoscere cosa farà il corpo

Un ciclo si scrive più facimente procedendo “dal basso verso l’alto”!

Si concepisce il programma nell'ordine inverso a quello della sua forma finale. Si scrivono nell'ordine:

- il corpo del ciclo;
- la condizione di controllo del ciclo;
- l'inizializzazione.

Infatti

prima si concepiscono le istruzioni del corpo del ciclo, in modo che producano un avvicinamento alla soluzione;

poi si scrive il test di controllo del ciclo, in modo che all'uscita dal ciclo si abbia la soluzione corretta;

per ultima si scrive l'inizializzazione, in modo da garantire una "partenza corretta".

Warning!

In alcuni esercizi d’esame vi chiederemo di identificare l’invariante di ciclo

→ Allenatevi a scrivere la I.C. delle vostre funzioni che contengono cicli

ESEMPI SUGLI ARRAY ORDINATI

Array ordinato?

Gli elementi dell'array hanno un tipo che consente di definire una **relazione d'ordine fra i valori**, esempi:

- ★ numeri con virgola: >, <, ≤, ≥
- ★ insiemi di valori: ⊂, ⊃
- ★ stringhe: ordine lessicografico

Date due posizioni qualsiasi nell'array, i valori degli elementi individuati rispetteranno la relazione d'ordine

Cercare un elemento in un array ordinato

Sfruttiamo la relazione d’ordine per trovare l’elemento di interesse più velocemente

- ★ Ricerca binaria

Ricerca binaria

```
DICHIARA DUE VARIABILI sn, dx PER DELIMITARE LA PORZIONE DI ARRAY arr[sn..dx]
IN CUI CERCARE;

MENTRE LA PORZIONE NON È VUOTA,

  DICHIARA UNA VARIABILE middle PER MEMORIZZARE L'INDICE A METÀ DI arr[sn..dx]

  SE el < arr[middle],

     ALLORA RESTRINGI LA PORZIONE A arr[sn..middle-1]

     ALTRIMENTI, SE arr[middle] < el,

                    ALLORA RESTRINGI LA PORZIONE A arr[middle+1..dx]

                    ALTRIMENTI RESTITUISCI middle;

RESTITUISCI -1.
```

Esempio

A	B	C	E	G	H	I	M
0	1	2	3	4	5	6	7

Array ordinato in ordine crescente

Vogliamo cercare la lettera H (**el == H**)

Esempio

A	B	C	E	G	H	I	M
0	1	2	3	4	5	6	**7**
SN							DX

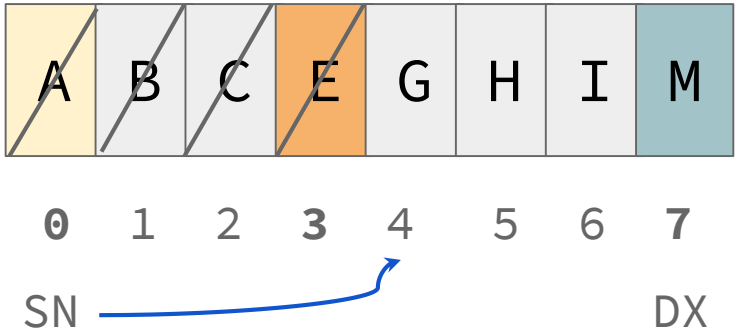
Esempio

A	B	C	E	G	H	I	M
0	1	2	**3**	4	5	6	**7**
SN							DX

MIDDLE = (7+0)/2 = 3

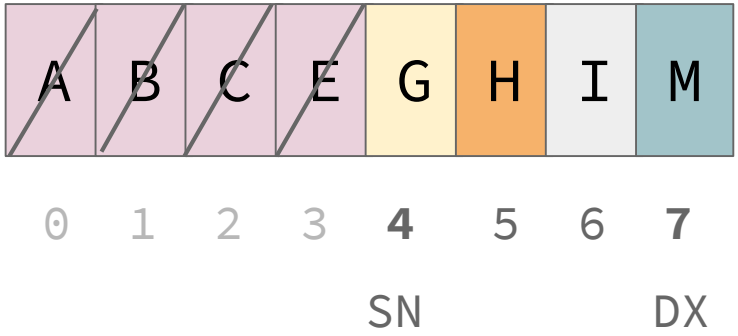
divisione intera

Esempio



array[MIDDLE] < EL (cioè E viene prima di H nell'alfabeto) ⟹ posso scartare tutti gli elementi da posizione 0 fino a MIDDLE

Esempio



MIDDLE = (4+7)/2 = 5

array[MIDDLE] == EL quindi si restituisce MIDDLE

Domande sulla ricerca binaria

- → Quale è l’invariante di ciclo della ricerca binaria?
- → Perché l’algoritmo è terminante?
- → Quanta fatica fa l’algoritmo nel caso peggiore?
  - → Quale è il caso peggiore?
  - → Quante iterazioni richiede l’algoritmo nel caso peggiore?

Ordinare un array

Dato un array di valori generalmente non ordinati, cambiare i posti di tali valori per rendere ordinato l’array

- ★ insertion sort
- ★ bubble sort
- ★ selection sort
- ★ quicksort

Insertion Sort

Questo algoritmo modifica l’array passato come parametro

P-IN(arr, n): n == lunghezza(a) ≥ 0

P-OUT(arr, n):

- ★ arr[0..N-1] è ordinato e
- ★ arr contiene gli stessi elementi di arr’

Insertion Sort

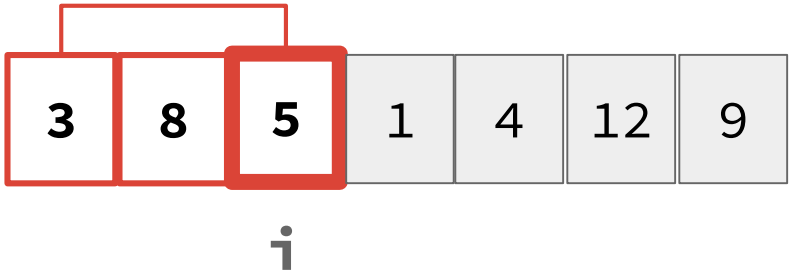
```
PER OGNI INDICE i DI a,
  INSERISCI arr[i] AL POSTO GIUSTO IN arr[0..i-1],
  SPOSTANDO DI 1 A DESTRA TUTTI GLI ELEMENTI > arr[i]
  PER FARE POSTO
```

Esempio

3	8	5	1	4	12	9

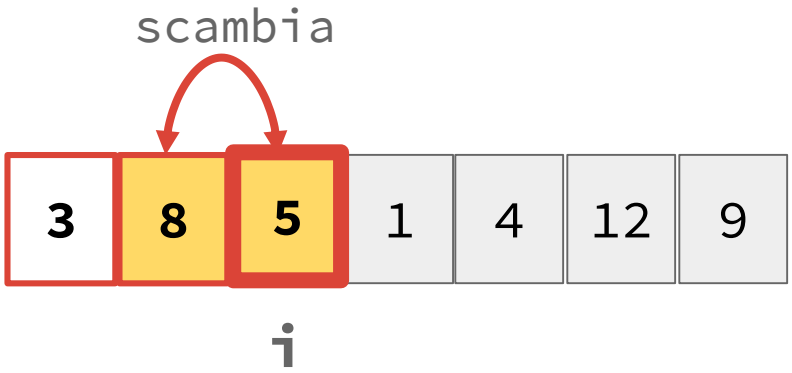
Supponiamo che arr contenga quanto nel disegno

Esempio



Al generico passo i di iterazione considereremo solo la porzione sinistra dell’array rispetto ad i

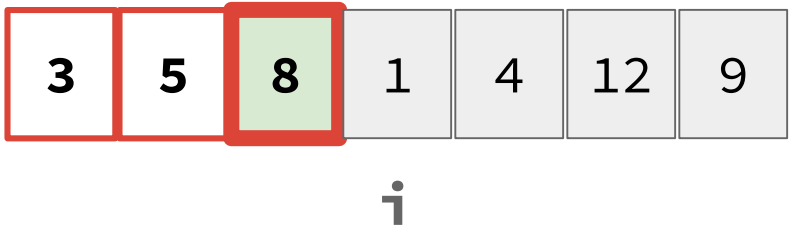
Esempio



confronteremo il valore alla posizione i con il predecessore (posizione i-1): se l'ordine non è rispettato scambieremo i valori delle due celle

Esempio

dopo lo scambio



poi ripeteremo finché il valore inizialmente alla cella di indice i (il valore 5 nell'esempio) non avrà trovato il suo posto

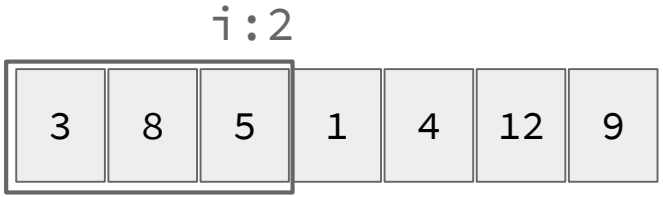
Insertion Sort: qualche passaggio

```
// deve ordinare tutti gli elementi dell'array
void insertionSort(int arr[], int n) {
    int i;
    // I.C.: a[0...i-1] ordinato    INVARIANTE DI CICLO
    for (i = 1; i < n; i++) { // PER CIASCUN INDICE i DI arr
        // INSERISCI a[i] AL POSTO GIUSTO IN a[0..i-1]
        insertion(arr, i);
    }
}
```

Insertion Sort: domanda

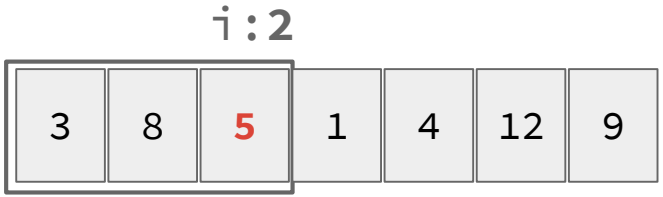
```
// deve ordinare tutti gli elementi dell'array
void insertionSort(int arr[], int n) {
    int i;
    // I.C.: a[0...i-1] ordinato
    for (i = 1; i < n; i++) {
        // PERCHÈ SI PARTE DA 1 E NON DA 0?
        insertion(arr, i);
    }
}
```

Insertion




```
void insertion(int arr[], int i) {
    int corrente = arr[i];  // valore
    int spostaqui = i;      // cursore
    while (spostaqui > 0 && arr[spostaqui - 1] > corrente) {
            arr[spostaqui] = arr[spostaqui-1];
            spostaqui --;
    }
    arr[spostaqui] = corrente;
}
```

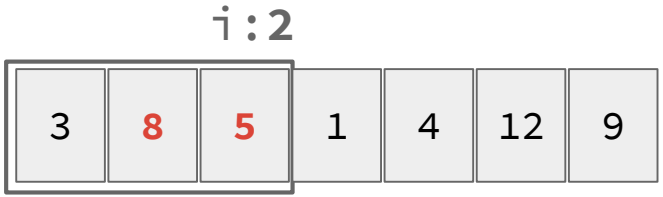
Insertion



corrente: 5
spostaqui: 2

```
void insertion(int arr[], int i) {
    int corrente = arr[i];  // valore
    int spostaqui = i;      // cursore
    while (spostaqui > 0 && arr[spostaqui - 1] > corrente) {
            arr[spostaqui] = arr[spostaqui-1];
            spostaqui --;
    }
    arr[spostaqui] = corrente;
}
```

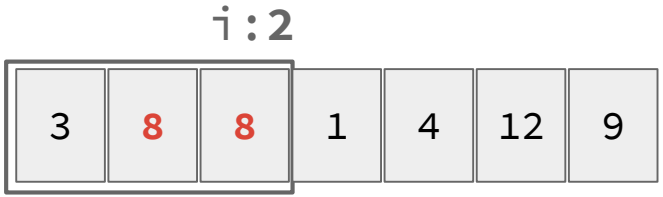
Insertion



corrente: 5
spostaqui: 2

```
void insertion(int arr[], int i) {
    int corrente = arr[i];  // valore
    int spostaqui = i;      // cursore
    while (spostaqui > 0 && arr[spostaqui - 1] > corrente) {
            arr[spostaqui] = arr[spostaqui-1];
            spostaqui --;
    }
    arr[spostaqui] = corrente;
}
```

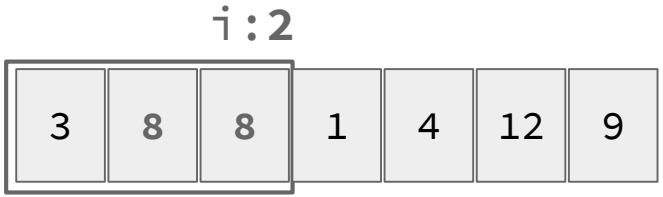
Insertion



corrente: 5
spostaqui: 2

```
void insertion(int arr[], int i) {
    int corrente = arr[i];  // valore
    int spostaqui = i;      // cursore
    while (spostaqui > 0 && arr[spostaqui - 1] > corrente) {
            arr[spostaqui] = arr[spostaqui-1];
            spostaqui --;
    }
    arr[spostaqui] = corrente;
}
```

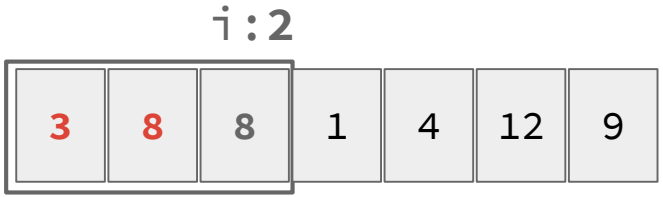
Insertion



corrente: 5
spostaqui: **1**

```
void insertion(int arr[], int i) {
    int corrente = arr[i];  // valore
    int spostaqui = i;      // cursore
    while (spostaqui > 0 && arr[spostaqui - 1] > corrente) {
            arr[spostaqui] = arr[spostaqui-1];
            spostaqui --;
    }
    arr[spostaqui] = corrente;
}
```

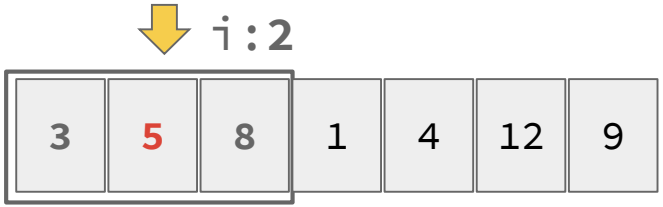
Insertion



corrente: 5
spostaqui: 1

```
void insertion(int arr[], int i) {
    int corrente = arr[i];  // valore
    int spostaqui = i;      // cursore
    while (spostaqui > 0 && arr[spostaqui - 1] > corrente) {
            arr[spostaqui] = arr[spostaqui-1];
            spostaqui --;
    }
    arr[spostaqui] = corrente;
}
```

Insertion



corrente: 5
spostaqui: 1

```
void insertion(int arr[], int i) {
    int corrente = arr[i];  // valore
    int spostaqui = i;      // cursore
    while (spostaqui > 0 && arr[spostaqui - 1] > corrente) {
            arr[spostaqui] = arr[spostaqui-1];
            spostaqui --;
    }
    arr[spostaqui] = corrente;
}
```

Funzione insertion

Invariante di ciclo:

P-IN-insertion(arr, i): arr[0..i-1] è ordinato

per rispettare questa assunzione la funzione viene richiamata su porzioni di array via via più grandi in cui l'unico elemento che può essere fuori posto è quello di indice i

P-OUT-insertion(arr, i): arr[0..i] ordinato e arr contiene gli stessi elementi di arr'

Quanta "fatica" fa l'insertion sort?

Questa funzione esegue due cicli, uno annidato nell'altro

Caso migliore: quando l'array è ordinato lo percorre una volta sola (il ciclo interno viene "saltato")

Caso peggiore: quando l'array è ordinato al contrario ogni elemento deve essere scambiato con tutti quelli che lo precedono

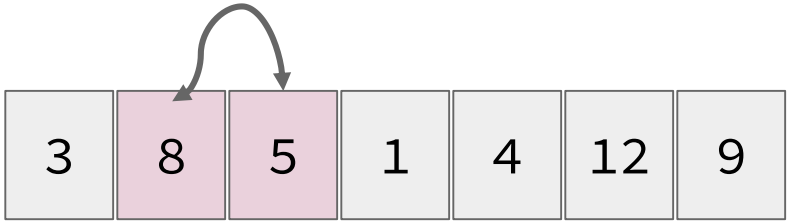
Bubble sort

3	8	5	1	4	12	9

percorre l’array soffermandosi su coppie di elementi consecutivi:

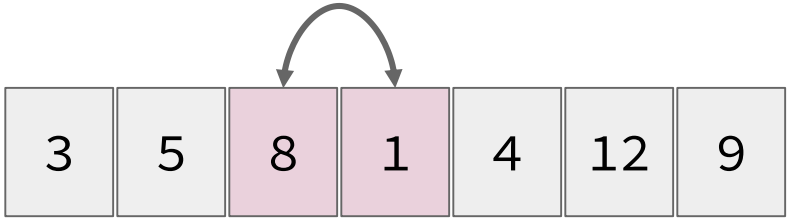
3 - 8: è ordinata, va bene così

Bubble sort



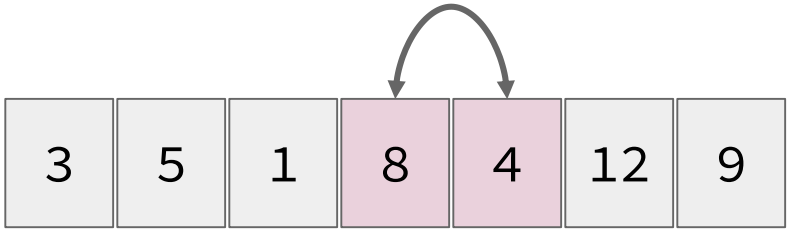
8 - 5: ordine violato, scambia i due valori

Bubble sort



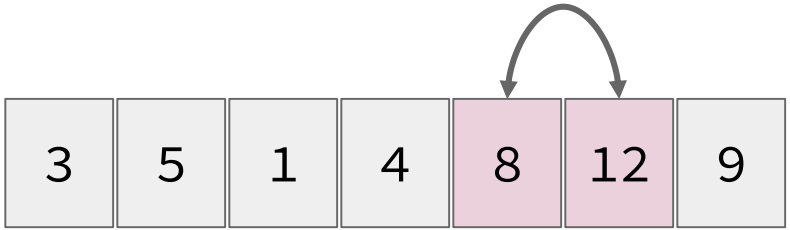
8 - 1: ordine violato, scambia i due valori

Bubble sort



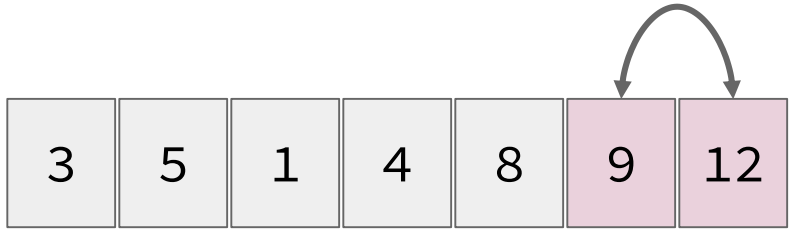
8 - 4: ordine violato, scambia i due valori

Bubble sort



8 - 12: ordine rispettato

Bubble sort



12 - 9: ordine violato, scambia i due valori

L’elemento più grande è venuto “a galla”, cioè ha raggiunto la sua posizione desiderata

Possono servire fino a N-1 ripetizioni del procedimento per ordinare l’array

Bubble sort: resto dell'esecuzione, simulazione

3	5	1	4	8	9	12

3	1	5	4	8	9	12

3	1	4	5	8	9	12

1	3	4	5	8	9	12

Bubble sort

P_IN: num. elem in arr == n > 0

```
void BubbleSort(int arr[], int n)
  // finché arr non è ordinato
  // percorri l’array scambiando arr[i] con arr[i-1]
  // se arr[i-1] > arr[i]
```

P_OUT:arr ordinato e arr contiene gli stessi elementi di arr’

Bubble sort

P_IN: num. elem in arr == n > 0

```
void BubbleSort(int arr[], int n)
  // finché arr non è ordinato
  for (int i=1; i<n; i++) {
      if (arr[i-1] > arr[i]) // scambia i due valori
  }
```

P_OUT:arr ordinato e arr contiene gli stessi elementi di arr’

Bubble sort

```
void BubbleSort(int arr[], int n)
  int temp;
  // finché arr non è ordinato
  for (int i=1; i<n; i++) {
      if (arr[i-1] > arr[i]) { // scambia i due valori
         valore = arr[i]:
         arr[i] = arr[i-1];
         arr[i-1] = valore;
      }
  }
```

Bubble sort

```
void BubbleSort(int arr[], int n)
  int temp;
  while ( CONDIZIONE ) { // finché arr non è ordinato
     for (int i=1; i<n; i++) {
         if (arr[i-1] > arr[i]) { // scambia i due valori
            … // visto prima
         }
  }
}
```

Bubble sort

```
void BubbleSort(int arr[], int n)
  int temp; int fine = 0;
  while ( !fine ) { // finché arr non è ordinato
     fine = 1; // è ottimista, prevede che finirà
     for (int i=1; i<n; i++) {
         if (arr[i-1] > arr[i]) { // scambia i due valori
            … fine = 0; // se avviene anche solo uno scambio fine torna a 0
         }
  }
}
```

Bubble sort: quanta "fatica" fa?

Anche in questo caso abbiamo due cicli annidati

Prestazioni simili a quelle dell'insertion sort

Domanda: Quale è l'invariante di ciclo di Bubble Sort?

Selection Sort

Assumo: arr è un array di lunghezza n > 0

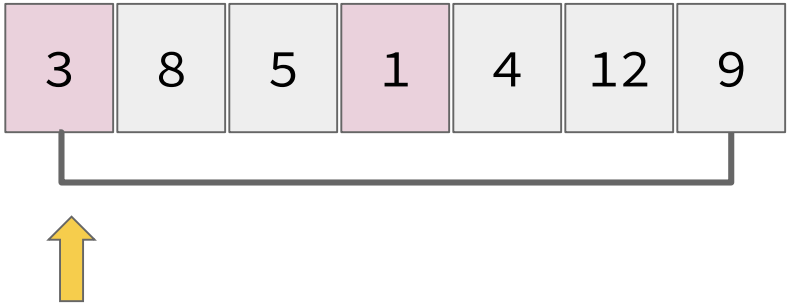
PER OGNI INDICE i DA 0 A n-2 RIPETI:

    CERCA L’ELEMENTO PIÙ PICCOLO DI arr[i..n]

    SCAMBIA QUESTO ELEMENTO CON L’ELEMENTO IN POSIZIONE i

P_OUT: arr è ordinato e contiene gli stessi elementi di arr’

Selection sort: simulazione

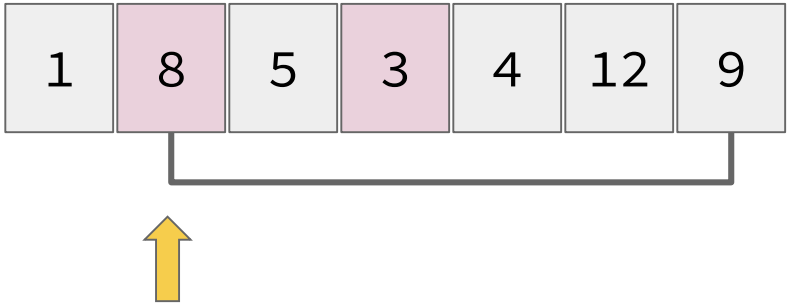


arr[0..6]: cerca il minimo

i == 0

scambia il minimo con l’elemento in posizione 0

Selection sort: simulazione

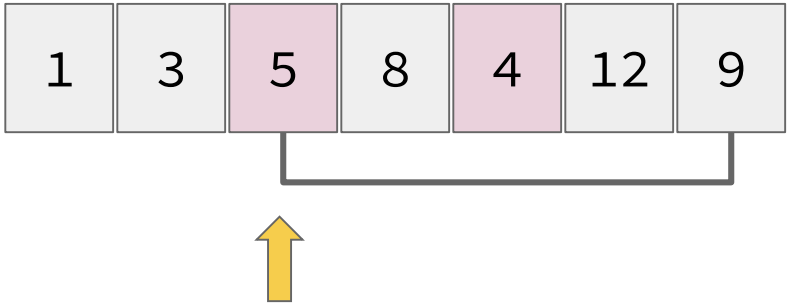


arr[1..6]: cerca il minimo

i == 1

scambia il minimo con l’elemento in posizione 1

Selection sort: simulazione

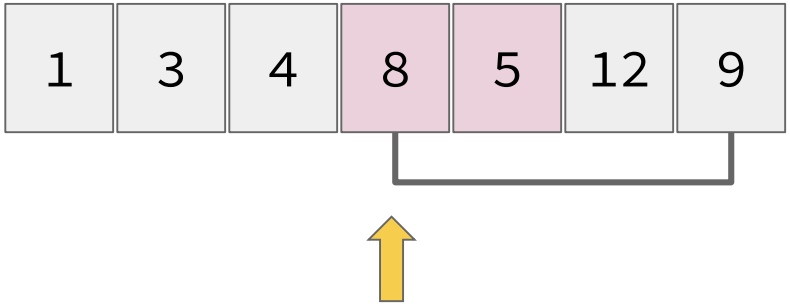


arr[2..6]: cerca il minimo

i == 2

scambia il minimo con l’elemento in posizione 2

Selection sort: simulazione

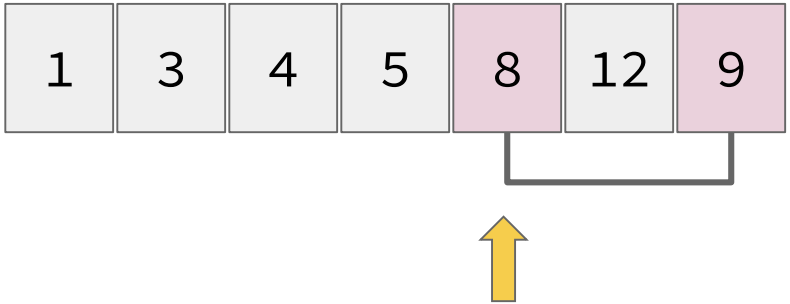


arr[3..6]: cerca il minimo

i == 3

scambia il minimo con l’elemento in posizione 3

Selection sort: simulazione



arr[4..6]: cerca il minimo

i == 4

scambia il minimo con l'elemento in posizione 4 (8 è già il minimo)

Selection sort: simulazione

arr[5..6]: cerca il minimo

i == 5

scambia il minimo con l’elemento in posizione 5

non serve andare oltre, rimane un solo elemento

Selection sort: risultato finale

1	3	4	5	8	9	12

Selection sort

```
selection_sort(int arr[], int n) {
  int i, j; // cursori per i due cicli annidati
  // PER OGNI INDICE i DA 0 A n-2 RIPETI:
  for (i=0; i<(n-1); i++) {
       // CERCA L'ELEMENTO PIÙ PICCOLO DI arr[i..n]
       // SCAMBIA QUESTO ELEMENTO CON L'ELEMENTO IN POSIZIONE i
  }
}
```

Selection sort

```
selection_sort(int arr[], int n) {
  int i, j;
  for (i=0; i<(n-1); i++) {
        // CERCA L'ELEMENTO PIÙ PICCOLO DI arr[i..n]
        for (j=i+1; j<n; j++)
                // confronta l'elem corrente con il minimo e aggiorna
        // SCAMBIA QUESTO ELEMENTO CON L'ELEMENTO IN POSIZIONE i
  }
}
```

Selection sort

```
selection_sort(int arr[], int n) {
  int i, j; int min;
  for (i=0; i<(n-1); i++) { // CERCA L'ELEMENTO PIÙ PICCOLO DI arr[i..n]
    min = i;
    for (j=i+1; j<n; j++)
        // confronta l'elem corrente con il minimo e aggiorna
        if (arr[j] < arr[min]) min = j;
     // SCAMBIA QUESTO ELEMENTO CON L'ELEMENTO IN POSIZIONE i
  }
}
```

Selection sort

```
selection_sort(int arr[], int n) {
  int i, j; int min;
  for (i=0; i<(n-1); i++) { // CERCA L'ELEMENTO PIÙ PICCOLO DI arr[i..n]
    min = i;
    for (j=i+1; j<n; j++)
        if (arr[j] < arr[min]) min = j;
    // SCAMBIA QUESTO ELEMENTO CON L'ELEMENTO IN POSIZIONE i
    scambia(arr, i, min);
  }
}
```

Selection sort: quanta fatica fa?

- ★ selection sort ha due cicli annidati
- ★ ma sposta ciascun elemento **al più una volta sola**

Domanda: Quale è l’invariante di ciclo di Selection Sort?

Selection sort: quanta fatica fa?

Provate a fare delle **simulazioni** (eseguite il codice a mano su vostri array):

- ★ array ordinato
- ★ array ordinato al contrario
- ★ array disordinato

Confrontate il comportamento di selection sort con quello del bubble sort e dell'insertion sort e annotate in una tabella le vostre osservazioni sui tre casi sperimentati

Quick sort

- Insertion sort, bubble sort e selection sort sono algoritmi di complessità media n^2 (dove n è il numero di elementi da ordinare)
- Esistono algoritmi di complessità media (ottima) $n \log n$, ad esempio:
  - Quick sort
  - Merge sort
- …entrambi basati sulla tecnica divide-and-conquer
- Vediamo più in dettaglio quicksort, l'algoritmo di ordinamento adottato nella libreria standard del C (funzione qsort)
- Nota: quicksort produce un ordinamento non stabile (vedi dopo)

quick Sort

Assumo: arr è un array di lunghezza n > 0

se n è pari a 1, return

altrimenti:

scegli un elemento dell’array come pivot (ad es: l’ultimo)

partiziona l’array in due sub-array:

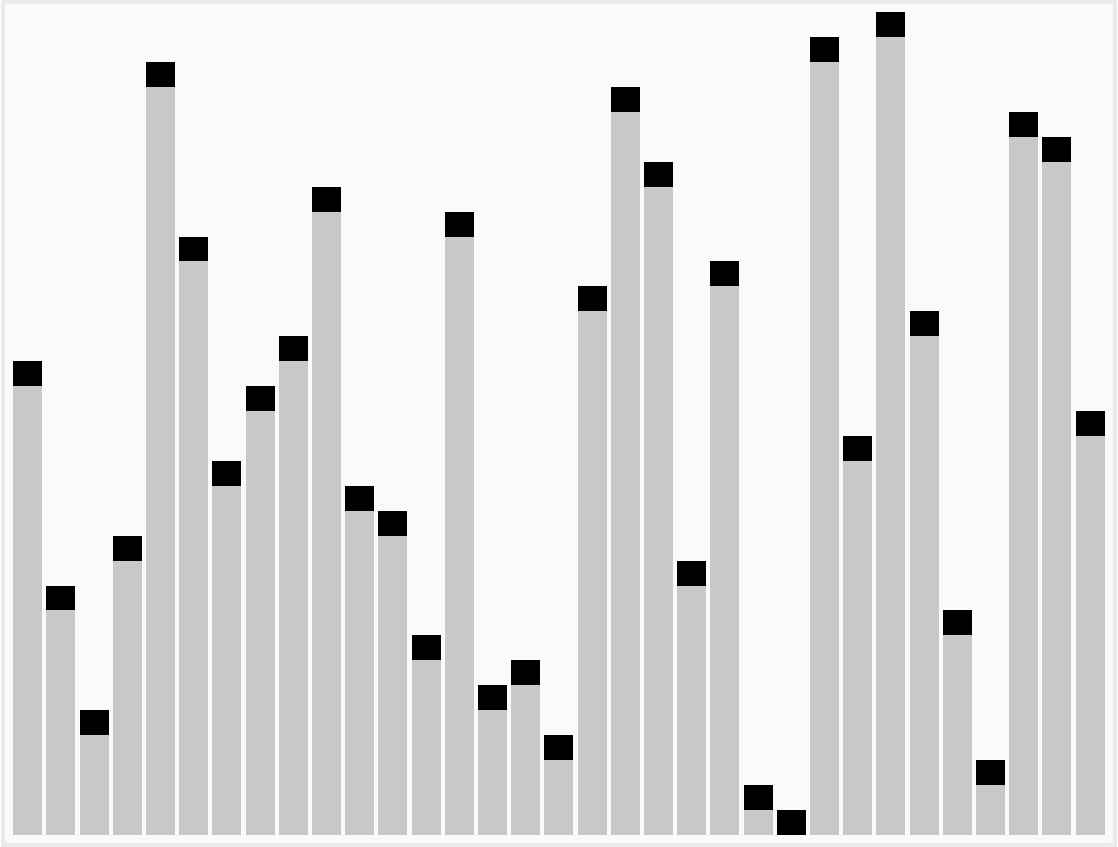
- gli elementi minori o uguali al pivot
- gli elementi maggiori del pivot

esegui quick sort sui due sub-array (doppia ricorsione!)

return

P_OUT: arr è ordinato e contiene gli stessi elementi di arr’

Quick sort



fonte: Wikipedia

Una DEFINIZIONE, una DOMANDA e una OSSERVAZIONE

Definizione (Stabilità di un algoritmo di ordinamento)

Un metodo di ordinamento si dice stabile se preserva l'ordine relativo degli elementi con chiavi[69] uguali all'interno della sequenza da ordinare.

[69] In molte applicazioni, quando si ordinano sequenze di struct, l'ordinamento è effettuato rispetto ad un solo membro (detto "chiave") – ad esempio, una sequenza di persone ordinate per data di nascita.

Una DEFINIZIONE, una DOMANDA e una OSSERVAZIONE

Definizione (Stabilità di un algoritmo di ordinamento)

Un metodo di ordinamento si dice stabile se preserva l'ordine relativo degli elementi con chiavi[69] uguali all'interno della sequenza da ordinare.

Quale tra i seguenti algoritmi di ordinamento (nella versione presentata nelle slides precedenti)

- `insertionSort`
- `selectionSort`
- `bubbleSort`

è stabilizzante?

[69] In molte applicazioni, quando si ordinano sequenze di struct, l'ordinamento è effettuato rispetto ad un solo membro (detto "chiave") – ad esempio, una sequenza di persone ordinate per data di nascita.

Una DEFINIZIONE, una DOMANDA e una OSSERVAZIONE

Definizione (Stabilità di un algoritmo di ordinamento)

Un metodo di ordinamento si dice stabile se preserva l'ordine relativo degli elementi con chiavi[69] uguali all'interno della sequenza da ordinare.

Quale tra i seguenti algoritmi di ordinamento (nella versione presentata nelle slides precedenti)

- `insertionSort`
- `selectionSort`
- `bubbleSort`

è stabilizzante?

OSSERVAZIONE (Sugli algoritmi di ordinamento stabili)

Per ordinare una sequenza S di tuple della forma $\langle key_1, \ldots, key_k, data_1, \ldots, data_m \rangle$ in ordine lessicografico rispetto alle chiavi $key_1, \ldots, key_k$ si può ordinarla – utilizzando un algoritmo stabile – prima rispetto a key_n (la chiave meno significativa), poi rispetto a key_{k-1}, poi ..., poi rispetto a key_2, e infine rispetto a key_1.

[69] In molte applicazioni, quando si ordinano sequenze di struct, l'ordinamento è effettuato rispetto ad un solo membro (detto "chiave") – ad esempio, una sequenza di persone ordinate per data di nascita.

Wikipedia: fonte ulteriore

su wikipedia trovate tutti questi algoritmi ed altri

MOLTO UTILE: per gli algoritmi di ordinamento trovate anche delle piccole animazioni che illustrano il procedimento

Copia-incollare le implementazioni non insegna nulla, per quanto più difficile sforzatevi di implementare le funzioni voi stessi